

A Control-Oriented Introduction to Policy-Based Reinforcement Learning

Ankit Goel

Abstract—This document provides a beginner-level introduction to reinforcement learning for students and researchers with a background in engineering and control systems. The fundamental concepts of policy optimization, exploration, reward functions, and policy-gradient methods are developed through simple illustrative examples. The assumed background includes linear algebra, gradient-based optimization, random variables, and probability density functions.

I. POLICY-BASED REINFORCEMENT LEARNING

Consider the discrete-time system

$$x_{k+1} = f(x_k, u_k), \quad (1)$$

where $x_k \in \mathbb{R}^n$ and $u_k \in \mathbb{R}^m$. Let $r_k = r(x_k, u_k)$ denote the reward obtained at time step k . The objective is to construct a control policy that maximizes the expected cumulative reward.

A. Parameterized Policy

Let $\mu_\theta: \mathbb{R}^p \rightarrow \mathbb{R}^m$ denote a parameterized control policy, where $\theta \in \mathbb{R}^p$ is a vector of policy parameters. Examples of policy μ_θ include linear controllers, radial-basis-function approximators, Gaussian-process policies, and neural networks.

B. Exploration Policy

To facilitate learning, a stochastic exploratory policy is employed. Note that $\mu_\theta(x_k)$ is the nominal policy output. The exploratory action is generated by setting

$$u_k = \mu_\theta(x_k) + \sigma \eta_k, \quad (2)$$

where $\eta_k \sim \mathcal{N}(0, I_m)$ and $\sigma > 0$ is the exploration variance. Equivalently, $u_k \sim \mathcal{N}(\mu_\theta(x_k), \sigma^2 I_m)$.

C. Trajectory Generation

Given an initial condition x_0 , the exploratory policy is applied to the system (1), where the control u_k is given by according to (2). This process generates a trajectory

$$\tau \triangleq \{x_0, u_0, x_1, u_1, \dots, x_N, u_N\}, \quad (3)$$

which is called an *episode*.

Ankit Goel is an Assistant Professor in the Department of Mechanical Engineering, University of Maryland, Baltimore County, 1000 Hilltop Circle, Baltimore, MD 21250. ankgoel@umbc.edu

D. Episode Return

The performance of an episode is quantified using the *return*

$$R(\tau) \triangleq \sum_{k=0}^N \gamma^k r_k, \quad (4)$$

where $\gamma \in (0, 1]$ is a discount factor. The objective of reinforcement learning is to determine policy parameters that maximize the expected return

$$J(\theta) \triangleq \mathbb{E}[R(\tau)]. \quad (5)$$

E. Policy Optimization

Policy optimization is the process of adjusting the policy parameters to increase the expected return given by 5. Numerous reinforcement-learning algorithms have been developed for this purpose, including policy-gradient methods, actor-critic methods, trust-region methods, and value-based methods. Although these approaches differ in how the policy parameters are updated, they all seek to improve policy performance using data collected from system trajectories. The present tutorial focuses on a simple policy-gradient method based on Monte Carlo estimates of the expected return.

1) *Policy Improvement*: To update the policy parameters θ , the reinforcement learning algorithm correlates random control perturbations with episode performance. The policy parameters are then updated to yield larger returns. Suppose that a perturbation sequence $\{\eta_0, \eta_1, \dots, \eta_N\}$ produces a trajectory with a large return. In this case, the policy parameters are modified so that similar control actions become more likely in future episodes. Conversely, perturbations that lead to poor performance are discouraged. This process gradually shifts the policy toward control actions that yield larger expected returns.

2) *Policy Gradient Approximation*: Let θ_j denote the policy parameters at the j th iteration. Suppose that M episodes are generated using the current policy parameterized by θ_j . Let τ_i , where $i = 1, \dots, M$, denote the resulting trajectories and let $R_i \triangleq R(\tau_i)$ denote the corresponding returns.

The objective of reinforcement learning is to maximize the expected return (5). The policy gradient is given by

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[R(\tau) \sum_{k=0}^{N-1} \nabla_{\theta} \log \pi_{\theta}(u_k | x_k) \right]. \quad (6)$$

Remark 1. In the case where $u_k \sim \mathcal{N}(\mu_{\theta}(x_k), \sigma^2 \mathbf{I}_m)$, the policy density is

$$\pi_{\theta}(u_k | x_k) = \frac{1}{(2\pi\sigma^2)^{m/2}} \exp \left(-\frac{\|u_k - \mu_{\theta}(x_k)\|_2^2}{2\sigma^2} \right). \quad (7)$$

Remark 2. Let $\pi_{\theta}(u|x)$ denote a probability density parameterized by θ . The exploratory control action is generated by sampling from this density, that is, $u_k \sim \pi_{\theta}(\cdot|x_k)$. Consequently, $\pi_{\theta}(u_k|x_k)$ denotes the probability density associated with the sampled action u_k at state x_k .

3) *Parameter Update:* The policy parameter θ_j are updated using the standard gradient *ascent* to maximize the cost (5), that is,

$$\theta_{j+1} = \theta_j + \alpha \hat{J}_{\theta}, \quad (8)$$

where $\alpha > 0$ is the learning rate. The procedure of trajectory generation, return evaluation, and parameter updating is repeated for a prescribed number of training iterations or until the observed improvement in the episode return becomes negligible.

F. Deployment

Let θ^* denotes the final trained parameter vector. The learned controller is then $u_k = \mu_{\theta^*}(x_k)$.

II. A MODULAR VIEW OF REINFORCEMENT LEARNING

Reinforcement learning a general framework for constructing decision-making systems that learn from interaction with an environment. The preceding section presented one particular reinforcement-learning algorithm based on a parameterized policy, Gaussian exploration, Monte Carlo return evaluation, and likelihood-ratio policy-gradient optimization. This algorithm is commonly known as the *REINFORCE* algorithm.

A reinforcement-learning algorithm is obtained by specifying how the policy is represented, how exploration is performed, how trajectories are generated, how performance is evaluated, and how the policy is updated. Different choices lead to different reinforcement-learning algorithms.

Consequently, most reinforcement-learning methods can be interpreted as combinations of the following six fundamental components.

1) Policy parameterization,

- 2) Exploration strategy,
- 3) Trajectory generation,
- 4) Return evaluation,
- 5) Policy optimization,
- 6) Deployment.

Figure 1 summarizes this modular perspective. Different reinforcement-learning algorithms can be viewed as different combinations of choices within these blocks.

Table I shows several popular RL algorithms. The first row corresponds to the REINFORCE algorithm presented in Section I. Note that different reinforcement-learning algorithms arise from different choices for exploration, performance evaluation, and optimization while retaining the same overall framework.

III. THE REINFORCE ALGORITHM APPLIED TO A SINGLE INTEGRATOR

To illustrate the modular reinforcement-learning framework introduced in Section II, consider the discrete-time single-integrator system

$$\dot{p} = u, \quad (9)$$

where $p \in \mathbb{R}^2$ is the position and the $u \in \mathbb{R}^2$ is the velocity. Using Euler's discretization, it follows that

$$p_{k+1} = p_k + \Delta t u_k, \quad (10)$$

where $p_k \triangleq p(k\Delta t) \in \mathbb{R}^2$, $u_k \triangleq u(k\Delta t) \in \mathbb{R}^2$ is the control input, and $\Delta t > 0$ is the sampling time. The objective is to move the system from an initial position p_0 to a desired goal position $p_g \in \mathbb{R}^2$. For this example, the six components of the reinforcement-learning framework are chosen as follows.

- 1) Policy parameterization: two-layer neural network,
- 2) Exploration strategy: Gaussian perturbation of the policy output,
- 3) Trajectory generation: simulation rollouts,
- 4) Return evaluation: discounted cumulative reward,
- 5) Policy optimization: REINFORCE policy-gradient update,
- 6) Deployment: fixed learned policy.

These choices correspond to the REINFORCE algorithm introduced in Section I.

A. Policy Parameterization

We consider a two-layer neural-network policy with one hidden layer. Let

$$a_k = W_1 e_k + b_1, \quad (11)$$

$$h_k = \tanh(a_k), \quad (12)$$

$$\nu_k = W_2 h_k + b_2, \quad (13)$$

where $e_k \triangleq p_k - p_g$ is the tracking error and $W_1 \in \mathbb{R}^{n_h \times 2}$, $b_1 \in \mathbb{R}^{n_h}$, $W_2 \in \mathbb{R}^{2 \times n_h}$, and $b_2 \in \mathbb{R}^2$ are

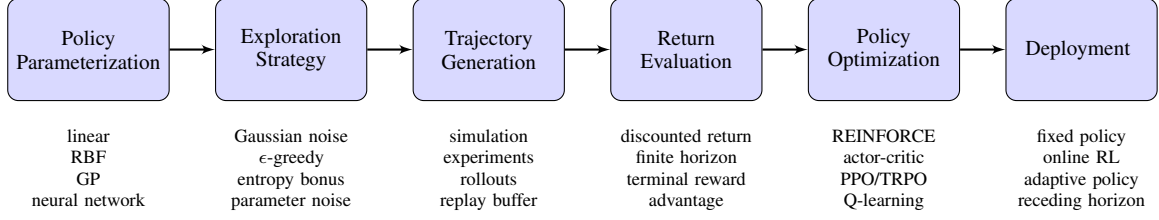


Fig. 1: A modular view of reinforcement-learning control. Different reinforcement-learning algorithms can be interpreted as different choices for policy parameterization, exploration, trajectory generation, return evaluation, and policy optimization.

Algorithm	Policy Representation	Exploration	Performance Evaluation	Optimization Method
REINFORCE	Neural Network	Gaussian Action Noise	Monte Carlo Return	Policy Gradient
Actor-Critic	Neural Network	Gaussian Action Noise	Value Function + Advantage	Policy Gradient + Critic Update
PPO	Neural Network	Gaussian Action Noise	Advantage Estimate	Clipped Policy Optimization
TRPO	Neural Network	Gaussian Action Noise	Advantage Estimate	Trust-Region Optimization
DDPG	Deterministic Neural Network	Action Noise	Q Function	Deterministic Policy Gradient
TD3	Deterministic Neural Network	Action Noise	Twin Q Functions	Deterministic Policy Gradient
SAC	Stochastic Neural Network	Entropy-Regularized Exploration	Soft Q Function	Soft Actor-Critic Update
DQN	Q Network	ϵ -Greedy	Temporal-Difference Target	Q-Learning
Double DQN	Q Network	ϵ -Greedy	Temporal-Difference Target	Double Q-Learning

TABLE I: Examples of reinforcement-learning algorithms viewed through the modular framework of Fig. 1.

the neural network parameters. The deterministic control policy used during deployment is

$$u_k = u_{\max} \tanh(\nu_k), \quad (14)$$

where $u_{\max} > 0$ is the control saturation limit.

B. Exploration Strategy

During training, exploration is introduced by perturbing the pre-saturation policy output. Specifically, the exploratory control action is

$$u_k = u_{\max} \tanh(\nu_k + \sigma \eta_k). \quad (15)$$

where $\eta_k \sim \mathcal{N}(0, I_2)$, and $\sigma > 0$ is the exploration standard deviation.

C. Trajectory Generation

Given an initial condition p_0 , the exploratory policy is applied to the system dynamics (10). This produces a trajectory

$$\tau \triangleq p_0, u_0, p_1, u_1, \dots, p_N, u_N, \quad (16)$$

which is referred to as an episode. Multiple episodes are generated during training using different realizations of the exploration noise. These trajectories provide the data used for policy improvement.

D. Return Evaluation

The performance of a trajectory is quantified through a reward function. At each time step,

$$r_k = -|e_k|_2^2. \quad (17)$$

If the trajectory reaches the goal region, that is, $|e_N|_2^2 < \epsilon$, then a terminal reward is added, that is,

$$r_N = -|e_N|_2^2 + R_{\text{goal}}, \quad (18)$$

where $\epsilon > 0$ is the goal tolerance and $R_{\text{goal}} > 0$ is the terminal reward. Finally, the performance of an episode is quantified by the discounted return

$$R = \sum_{k=0}^N \gamma^k r_k, \quad (19)$$

where $\gamma \in (0, 1]$ is the discount factor.

E. Policy Optimization

The policy is optimized using the REINFORCE algorithm. Since exploration perturbations are sampled from $(\nu_k, \sigma^2 I_2)$, the corresponding log-likelihood is

$$\pi_{\theta}(\tilde{\nu}_k | e_k) = \frac{1}{(2\pi\sigma^2)} \exp\left(-\frac{|\tilde{\nu}_k - \nu_k|_2^2}{2\sigma^2}\right), \quad (20)$$

where $\tilde{\nu}_k \triangleq \nu_k + \sigma\eta_k$. Consequently,

$$\log \pi_\theta(\tilde{\nu}_k | e_k) = -\frac{|\tilde{\nu}_k - \nu_k|_2^2}{2\sigma^2}. \quad (21)$$

For a batch of M episodes, let R_i denote the return of the i th episode. Define the normalized advantage

$$A_i = \frac{R_i - \bar{R}}{s_R + \varepsilon_R}, \quad (22)$$

where

$$\bar{R} = \frac{1}{M} \sum_{i=1}^M R_i, \quad (23)$$

and s_R is the sample standard deviation of the episode returns.

The policy parameters are updated according to

$$\theta_{j+1} = \theta_j + \alpha \frac{1}{M} \sum_{i=1}^M A_i \sum_{k=0}^{N-1} \frac{\partial}{\partial \theta} \log \pi_\theta(\tilde{\nu}_{i,k} | e_{i,k}). \quad (24)$$

where $\alpha > 0$ is the learning rate.

F. Deployment

After training, the exploration noise is removed and the learned controller is deployed as

$$u_k = u_{\max} \tanh(W_2 \tanh(W_1 e_k + b_1) + b_2), \quad (25)$$

where

$$e_k = p_k - p_g. \quad (26)$$

The policy parameters $\theta = \{W_1, b_1, W_2, b_2\}$ remain fixed during deployment and are not updated online.